SkillNavora

MODULE 8 HANDBOOK

Module 8: Cloud Computing, DevOps and Production Deployment

8 sections, every topic explained with how it works, examples and resources.

From Non-Coder to Enterprise AI Builder: Learn, Build, Deploy, Get Hired and Launch Your AI Business.

Varun Kumar · SkillNavora · skillnavora.in

Edition: November 2026. Links point to official documentation where available; check them for the latest versions.

Financial figures are hypothetical examples, not advice.

Module 8: Cloud Computing, DevOps and Production Deployment

Every section below explains the concept, how it works, each listed topic in detail, a worked example and where to learn more.

Sections

1. 8.1 Cloud Fundamentals
2. 8.2 Cloud Platforms
3. 8.3 Containerisation
4. 8.4 Kubernetes
5. 8.5 CI/CD
6. 8.6 Advanced Cloud Architecture
7. 8.7 Production AI Deployment
8. 8.8 Cloud Security

MODULE 8 · SECTION 8.1

8.1 Cloud Fundamentals

DEFINITION

> Cloud computing rents computing power, storage, networking and managed services over the internet, paid by use, instead of buying and running your own servers.

HOW IT WORKS

- IaaS gives raw virtual machines and networks (you manage the OS). PaaS runs your code on a managed platform (you manage the app). SaaS is finished software you simply use.
- Public cloud is shared provider infrastructure; private cloud is dedicated to one organisation; hybrid mixes the two.
- A region is a geographic location; availability zones are separate data centres inside a region. Spreading across zones survives one data-centre failure.
- The four building blocks are compute, storage, networking and databases.

EVERY TOPIC EXPLAINED

What cloud computing is. Cloud computing delivers computing resources on demand over the internet with pay-as-you-go pricing, instead of owning hardware. Benefits: speed, elasticity, global reach and managed services. Trade-offs: cost control, vendor lock-in and shared responsibility for security.

IaaS, PaaS and SaaS. IaaS: you rent virtual machines, storage and networks (EC2, Azure VMs, Compute Engine). PaaS: you deploy code and the platform runs it (App Service, Cloud Run, Elastic Beanstalk). SaaS: complete applications you use (Gmail, Salesforce).

Public, private and hybrid cloud; regions and availability zones. Public cloud: shared provider infrastructure. Private cloud: dedicated to one organisation. Hybrid cloud: on-premises plus public cloud. Regions are geographic areas (Mumbai, Frankfurt); availability zones are isolated data centres within a region for resilience; choose regions for latency and data residency rules.

Compute, storage, networking, databases. Compute runs code (VMs, containers, serverless). Storage keeps files (object storage such as S3), disks (block) and shared folders (file). Networking connects resources (virtual networks, subnets, load balancers, DNS). Databases are offered as managed services so the provider handles backups and patching.

WORKED EXAMPLE

> A startup runs its API on a PaaS (Cloud Run or App Service), keeps files in object storage, and uses a managed PostgreSQL database in two availability zones.

RESOURCES

- AWS: what is cloud computing https://aws.amazon.com/what-is-cloud-computing/
- Google Cloud documentation https://cloud.google.com/docs

8.2 Cloud Platforms

DEFINITION

> AWS, Microsoft Azure and Google Cloud Platform are the three largest public clouds. They offer similar building blocks under different names.

HOW IT WORKS

- Identity and Access Management (IAM) controls who can do what. Give each person and service only the permissions it needs.
- Equivalent services: compute (EC2 / Virtual Machines / Compute Engine), serverless containers (App Runner / Container Apps / Cloud Run), object storage (S3 / Blob Storage / Cloud Storage), managed PostgreSQL (RDS / Azure Database / Cloud SQL).
- AI services: Amazon Bedrock, Azure OpenAI and AI Foundry, Google Vertex AI give managed access to models inside your cloud account.
- Cost management: set budgets and alerts on day one; delete unused resources.

EVERY TOPIC EXPLAINED

AWS, Microsoft Azure and Google Cloud Platform foundations. AWS: the largest catalogue of services, widely used by startups and enterprises. Microsoft Azure: strong with enterprises using Microsoft 365, Active Directory and Azure OpenAI. Google Cloud Platform: strong in data, analytics, Kubernetes (GKE) and Vertex AI with Gemini.

Accounts, IAM, compute, storage, managed databases, AI services. Account setup: enable MFA on the root account, create admin users, set budget alerts. IAM: users, groups, roles and policies grant least-privilege permissions. Compute: EC2/VMs/Compute Engine, serverless (Lambda, Azure Functions, Cloud Functions), containers (ECS/EKS, AKS, GKE, Cloud Run). Storage: S3, Blob Storage, Cloud Storage. Managed databases: RDS, Azure Database for PostgreSQL, Cloud SQL. AI services: Bedrock, Azure OpenAI/AI Foundry, Vertex AI.

Networking, monitoring, security, cost management, deployment. Networking: VPC/VNet, subnets, security groups, private endpoints. Monitoring: CloudWatch, Azure Monitor, Cloud Monitoring. Security: KMS, secrets managers, security centres. Cost management: budgets, cost explorer, tagging, right-sizing and shutting down idle resources. Deployment: console for learning, Infrastructure as Code and CI/CD for real work.

WORKED EXAMPLE

> An enterprise customer requires data to stay in their Azure tenant, so the FDE deploys the app to Azure Container Apps and calls models through Azure's AI service.

RESOURCES

- AWS documentation https://docs.aws.amazon.com/
- Microsoft Azure documentation https://learn.microsoft.com/azure/
- Google Cloud training: AI and ML https://cloud.google.com/training/machinelearning-ai

8.3 Containerisation

DEFINITION

> A container packages an application with everything it needs to run (code, runtime, libraries, settings), so it runs the same on a laptop and in production.

HOW IT WORKS

- A Dockerfile lists the build steps. docker build turns it into an image; docker run starts a container from that image.
- Virtual machines include a full operating system; containers share the host kernel, so they start in seconds and use less memory.
- Docker Compose starts several containers together (for example API + database + cache) from one YAML file.
- Images are pushed to a registry (Docker Hub, ECR, Artifact Registry) so servers can pull them.

EVERY TOPIC EXPLAINED

What containers are; virtual machines vs containers. A container is a lightweight, isolated package of an app and its dependencies that runs consistently anywhere. It solves 'it works on my machine'.

Docker images, containers, Dockerfiles, Docker Compose. A virtual machine virtualises hardware and runs a full guest operating system (heavy, minutes to boot). Containers share the host OS kernel and isolate processes (light, seconds to start). Docker images are read-only templates built in layers from a Dockerfile (FROM, WORKDIR, COPY, RUN, EXPOSE, CMD); containers are running instances of images.

Container networking and registries. Docker Compose defines multi-container apps in docker-compose.yml (services, ports, volumes, environment) and starts them with docker compose up. Container networking lets services reach each other by name (api can call db:5432). Registries (Docker Hub, Amazon ECR, Azure Container Registry, Google Artifact Registry) store and version images with tags.

WORKED EXAMPLE

> FROM python:3.12-slim / COPY . /app / RUN pip install -r requirements.txt / CMD ["uvicorn", "main:app", "--host", "0.0.0.0"] builds a FastAPI image.

RESOURCES

- Docker: get started (official) https://docs.docker.com/get-started/
- Docker Compose docs https://docs.docker.com/compose/

8.4 Kubernetes

DEFINITION

> Kubernetes runs and manages containers across a cluster of machines: it starts them, restarts them when they fail, scales them and routes traffic to them.

HOW IT WORKS

- A cluster has nodes (machines). A Pod is the smallest unit and holds one or more containers.
- A Deployment keeps a set number of identical Pods running and performs rolling updates and rollbacks.
- A Service gives a group of Pods one stable address (types: ClusterIP, NodePort, LoadBalancer).
- ConfigMaps hold settings; Secrets hold sensitive values; the Horizontal Pod Autoscaler adds Pods when load rises.

EVERY TOPIC EXPLAINED

Clusters, nodes, pods, deployments, services. Kubernetes (K8s) orchestrates containers across machines. A cluster has a control plane (API server, scheduler, controllers, etcd) and worker nodes. Pods are the smallest unit (one or more containers sharing network and storage). Deployments declare the desired number of Pod replicas and roll out updates. Services give Pods a stable network name and load-balance between them.

ConfigMaps, secrets, scaling, monitoring, orchestration. ConfigMaps hold non-secret configuration; Secrets hold sensitive values (base64-encoded, so also encrypt at rest and restrict access). Scaling: manually (kubectl scale) or automatically with the Horizontal Pod Autoscaler. Monitoring: metrics-server, Prometheus, Grafana and logs. Orchestration means Kubernetes keeps reality matching your declared YAML: restarting failed Pods and rescheduling them on healthy nodes.

WORKED EXAMPLE

> A Deployment with 3 replicas of the API sits behind a LoadBalancer Service; when CPU passes 70% the autoscaler grows it to 6 Pods.

RESOURCES

- Kubernetes concepts (official) https://kubernetes.io/docs/concepts/
- Kubernetes basics tutorial https://kubernetes.io/docs/tutorials/kubernetes-basics/

8.5 CI/CD

DEFINITION

> Continuous integration automatically builds and tests every change. Continuous delivery keeps the software always ready to release; continuous deployment releases each passing change automatically.

HOW IT WORKS

- A pipeline runs on each push: install, lint, test, build image, push image, deploy.
- GitHub Actions workflows are YAML files in .github/workflows; the 'on' key sets the trigger, 'jobs' hold 'steps'.
- Failing tests stop the pipeline. Rollback re-deploys the previous image if a release breaks production.

EVERY TOPIC EXPLAINED

Continuous integration, delivery and deployment. Continuous integration: every push is built and tested automatically, catching problems early. Continuous delivery: every passing build is ready to deploy with one approval. Continuous deployment: every passing build goes to production automatically.

GitHub Actions; build, testing and deployment pipelines. GitHub Actions runs workflows defined in .github/workflows/*.yml: triggers (on: push, pull_request), jobs, runners (ubuntu-latest), steps (uses: actions/checkout, run: pytest). Build pipelines compile and package; testing pipelines run unit, integration and evaluation tests; deployment pipelines push images and update the environment, using stored secrets.

Automated testing, rollbacks, release management. Automated testing gates the pipeline: no deploy if tests fail. Rollbacks restore the previous version quickly (redeploy the previous image tag or use Kubernetes rollout undo). Release management uses version tags, release notes, staged rollouts (staging → production) and feature flags.

WORKED EXAMPLE

> On every pull request, a workflow runs pytest; on merge to main it builds a Docker image and deploys it to Cloud Run.

RESOURCES

- GitHub Actions workflow syntax (official) https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax
- GitHub Actions documentation https://docs.github.com/en/actions

8.6 Advanced Cloud Architecture

DEFINITION

> Advanced patterns help systems scale and stay maintainable: serverless functions, microservices, event-driven design, API gateways and infrastructure as code.

HOW IT WORKS

- Serverless runs code on demand and bills per call; good for spiky or small workloads.
- Microservices split a system into independently deployed services; event-driven systems communicate through queues or topics so producers and consumers are decoupled.
- An API gateway is the single entry point that handles auth, rate limits and routing; a load balancer spreads traffic across instances.
- Infrastructure as Code (Terraform, Helm) describes infrastructure in versioned files: write, plan, apply.

EVERY TOPIC EXPLAINED

Serverless, microservices, event-driven architecture, API gateways, load balancing. Serverless: functions run on events and scale to zero (Lambda, Cloud Functions). Microservices: small independently deployable services owning their data. Event-driven architecture: services publish and subscribe to events through queues or streams (SQS, Pub/Sub, Kafka). API gateways: one front door with auth, rate limits and routing. Load balancing: spreads traffic across healthy instances.

Infrastructure as Code: Terraform, Helm; cloud networking; identity and secrets management. Infrastructure as Code: Terraform describes cloud resources in HCL; the workflow is write → terraform plan (preview) → terraform apply; state tracks what exists. Helm packages Kubernetes YAML into charts with values per environment. Cloud networking: VPCs, private subnets, NAT, peering, private endpoints. Identity management: SSO, IAM roles, workload identity. Secrets management: Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager.

WORKED EXAMPLE

> Document uploads publish an event to a queue; a serverless function extracts text and another indexes it, each scaling on its own.

RESOURCES

- Terraform tutorials (official) https://developer.hashicorp.com/terraform/tutorials
- Helm documentation https://helm.sh/docs/

8.7 Production AI Deployment

DEFINITION

> Production AI deployment runs model-based features reliably for real users, with monitoring of quality, latency and cost as well as uptime.

HOW IT WORKS

- Separate environments (dev, staging, production) with their own keys and data.
- Log every model call with prompt version, model version, latency, tokens and cost; trace multi-step agent runs end to end (OpenTelemetry).
- Set timeouts, retries with backoff, and fallbacks to a second model or a safe message.
- Version prompts and models, run the evaluation set before each release, and keep a disaster-recovery plan.

EVERY TOPIC EXPLAINED

Model serving; AI inference endpoints; environment management. Model serving for AI apps: either call a hosted API or host models yourself (vLLM, TGI, SageMaker, Vertex endpoints). AI inference endpoints need timeouts, retries and fallbacks. Environment management keeps dev, staging and production separate, each with its own keys, models and data.

Monitoring, logging, tracing, latency and cost monitoring. Monitoring: request rates, errors, latency (p50/p95), token usage. Logging: prompts, outputs, tool calls and versions, with personal data masked. Tracing: follow one request through every step (OpenTelemetry, LangSmith, Langfuse). Latency monitoring finds slow steps; cost monitoring tracks spend per feature, customer and model.

Model versioning, scaling, reliability, disaster recovery. Model versioning pins exact model versions and records prompt versions. Scaling: autoscale app servers, use queues for heavy jobs, cache repeated answers, and respect provider rate limits. Reliability: fallback models, graceful degradation and health checks. Disaster recovery: backups, multi-zone deployment, a tested restore runbook and stated recovery time.

WORKED EXAMPLE

> A dashboard shows p95 latency 2.1 s, cost $0.004 per request and faithfulness 93% this week; an alert fires if cost per request doubles.

RESOURCES

- OpenTelemetry documentation https://opentelemetry.io/docs/
- Google SRE book (free) https://sre.google/sre-book/table-of-contents/

8.8 Cloud Security

DEFINITION

> Cloud security protects identities, data and networks in the cloud through least privilege, encryption, isolation and monitoring.

HOW IT WORKS

- Least privilege: each user and service gets only the permissions it needs. Use roles, not shared keys.
- Encrypt data at rest and in transit (TLS). Store secrets in a secrets manager, never in code or Git.
- Isolate networks (private subnets, security groups) and tenants' data. Scan images and dependencies for vulnerabilities.
- Monitor audit logs and alerts; map controls to the compliance regime the customer needs.

EVERY TOPIC EXPLAINED

Authentication, authorisation, encryption, network security, data isolation. Authentication verifies identity (SSO, MFA). Authorisation grants permissions (RBAC, IAM policies). Encryption protects data in transit (TLS/HTTPS) and at rest (KMS-managed keys). Network security uses private networks, firewalls/security groups and WAFs. Data isolation keeps tenants and environments separate.

Secrets management, least privilege, vulnerability management, security monitoring, compliance. Secrets management stores keys in a vault and rotates them. Least privilege gives each identity only what it needs. Vulnerability management scans code, dependencies and images (Dependabot, Trivy) and patches promptly. Security monitoring collects audit logs and alerts on unusual activity. Compliance maps controls to standards such as ISO 27001, SOC 2, GDPR and India's DPDP Act.

WORKED EXAMPLE

> An API key accidentally committed to GitHub is caught by secret scanning; it is revoked, rotated and moved into the cloud secrets manager.

RESOURCES

- OWASP Cheat Sheet Series https://cheatsheetseries.owasp.org/
- AWS Well-Architected: security pillar https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html